

Program-Level Proof Rules for Almost-Sure Termination of Probabilistic Sampling Algorithms

Anonymous author(s)

Abstract. Almost-sure termination (AST) is essential to the total correctness of probabilistic sampling algorithms. Majumdar and Sathiyarayan provide a sound and relatively complete AST rule for probabilistic control-flow graphs (pCFGs), whose application to source programs requires a graph translation and certificates over control locations and valuations. We lift their two-certificate approach to a rule stated directly over the distribution-transformer semantics of probabilistic while-loops with loop-free bodies. A non-negative supermartingale controls escape to unbounded regions, while a natural-valued variant decreases with uniformly positive probability and is bounded on each supermartingale sublevel set. Both obligations concern complete loop iterations, so users supply certificates over source states without intermediate control-location annotations. We establish soundness and relative completeness for this program class through certificate translations between the two levels.

We illustrate the rule on the one-dimensional random walk, then give both pCFG-level and program-level AST proofs of the Fast Dice Roller. This comparison makes explicit the location-dependent cases and offsets eliminated by reasoning over complete iterations. Further program-level proofs cover rejection and restart in the Fast Loaded Dice Roller and dyadic refinement in the Han-Hoshi sampler. For FDR and FLDR, existing distributional invariants can be reused to integrate these termination arguments with output-correctness proofs. The case studies demonstrate how the rule exposes the probabilistic progress argument while reducing control-flow bookkeeping.

Keywords: almost-sure termination · probabilistic programs · sampling algorithms · distribution transformers · loop invariants · supermartingales · Fast Dice Roller

1 Introduction

Randomized algorithms are routinely used in simulation, machine learning, network protocols, and exact sampling. Their correctness has two independent dimensions. *Partial correctness* states that, whenever an algorithm returns, its result has the intended distribution. *Almost-sure termination* (AST) states that it returns with probability one. Only their combination establishes total correctness: without AST, a sampler may produce the right distribution on terminating runs while losing probability mass to divergence.

AST proofs are subtle. A probabilistic loop may terminate almost surely even though its expected runtime is infinite and even though no globally bounded ranking function exists. The symmetric random walk on the non-negative integers is the canonical example: from $x > 0$, move to $x - 1$ or $x + 1$ with equal probability and stop at zero. It reaches zero with probability one, yet its expected hitting time is infinite. Classical finite-state or bounded-variant arguments therefore do not suffice.

Majumdar and Sathiyarayan recently introduced a sound and relatively complete rule for probabilistic termination at the level of probabilistic control-flow graphs (pCFGs) [?]. Their rule combines a supermartingale with a natural-valued variant. The supermartingale controls the probability of escaping to regions in which the variant becomes large; within each of its sublevel sets, the variant is bounded and decreases with a uniformly positive probability. This is a strong semantic characterization of AST.

Applying this pCFG rule to a source program introduces additional proof structure. A loop iteration that is conceptually a single sampling step becomes a collection of locations and transitions. The two certificate functions take location-valuation pairs as arguments, often requiring different symbolic cases for assignments, guards, and probabilistic branches. For the Fast Dice Roller (FDR), for example, a simple source-level observation about the growth of $v - c$ turns into a location-indexed variant with several cases. This overhead concerns the representation of the certificates, rather than a need to enumerate individual valuations. It is especially unfortunate for exact samplers, whose functional correctness is already naturally expressed by *distributional invariants* and distribution-transformer semantics [?].

Our approach. We lift the two-certificate pCFG rule to the source-program level, summarizing the complete effect of one loop-body execution by its post-distribution. Let $C = \text{while } (b) \{B\}$, where B is loop-free. The rule requires:

- (i) a distributional invariant I closed under one loop iteration;
- (ii) a non-negative supermartingale v whose expected value does not increase after executing B ;
- (iii) a natural-valued variant u that, with probability at least $\epsilon > 0$, is smaller after executing B ; and
- (iv) boundedness of u in every sublevel set of v .

All obligations are stated over source states and the distribution transformer $\text{sp}[[B]]$. The rule therefore reasons at the semantic granularity at which programmers understand a probabilistic loop: one iteration at a time.

Source-level AST rules already exist, notably the rule of McIver et al. [?]. Our contribution is a program-level formulation of the separate supermartingale and variant certificates of Majumdar and Sathiyarayan, together with its metatheory and integration with distributional invariants. The separation itself comes from the pCFG rule; the new formulation makes that approach available through obligations over a complete source-level iteration.

Why sampling algorithms? Exact sampling algorithms provide a natural setting in which to evaluate AST proof rules. Their loop invariants often describe entire distributions rather than merely state predicates; rejection sampling introduces restarts that defeat naïve pointwise descent arguments; and total correctness requires the AST proof to compose with a pre-existing output-distribution proof. Our formulation directly supports this workflow. Reachability information is extracted from the support of a distributional invariant, while the richer equal-mass constraints used for partial correctness can be reused unchanged.

After a one-dimensional random walk introduces the rule’s mechanism, we study three samplers with complementary roles. First, the Fast Dice Roller (FDR) [?] samples uniformly from $\{0, \dots, n - 1\}$ using fair bits. We give both pCFG-level and program-level proofs, making the additional location-dependent cases and offsets explicit. This comparison demonstrates a reduction in the bookkeeping needed to express the same termination argument. Second, the Fast Loaded Dice Roller (FLDR) [?] samples from a finite distribution with rational probabilities using a compact decision tree and rejection. Its program-level proof handles progress and reset in one body step. Third, the Han–Hoshi sampler repeatedly refines a dyadic interval until it lies entirely within a target-distribution interval. Its certificate counts the target boundaries inside the current interval. FLDR and Han–Hoshi demonstrate applicability through program-level proofs; the explicit comparison between the two proof rules is provided by FDR. The resulting evidence concerns certificate structure and manual proof obligations, without a measured claim about proof-development or solver runtime.

For FDR and FLDR, prior work already supplies distributional invariants and AST arguments [?]. We reuse these verification problems to show how a general program-level AST rule fits the existing distributional proof framework.

Contributions. This paper makes the following contributions.

1. We present a new program-level proof rule for AST of probabilistic while-loops with loop-free bodies. The rule is phrased entirely in distribution-transformer semantics and cleanly separates the supermartingale and variant obligations (Section 4).
2. We establish soundness by compiling program-level certificates into certificates for the relatively complete pCFG rule. Conversely, we derive program-level certificates from pCFG certificates, yielding relative completeness with respect to the underlying first-order arithmetic theory (Section 5).
3. We illustrate the mechanism on the one-dimensional random walk and explain how the source-level rule removes location-dependent bookkeeping (Section 7).
4. We give parallel pCFG- and program-level AST proofs for FDR. The comparison exhibits the location-dependent certificate cases and offsets eliminated at program level, and shows how to reuse an existing distributional invariant (Section 8.1).
5. We prove AST of FLDR and Han–Hoshi at program level. FLDR captures rejection and restart through one body-step argument over its decision tree,

while Han–Hoshi uses a certificate based on the target-distribution boundaries contained in the current dyadic interval. These case studies demonstrate the rule’s applicability to distinct sampling mechanisms. For FDR and FLDR, the AST proofs combine with existing distributional invariants to recover total correctness (Sections 8.2 and 8.3).

Scope. The present rule targets a single while-loop with a loop-free body and no nondeterministic choice. This fragment already captures the characteristic structure of many sampling algorithms, where all potentially unbounded computation is concentrated in one main loop, while preprocessing and the individual loop iterations terminate. Restricting the body to be loop-free therefore still covers a broad and practically relevant class of samplers, while making each iteration a finite distribution-transformer step and keeping the correspondence to pCFG paths transparent.

The restriction also provides a natural basis for treating nested loops compositionally. In particular, an inner loop can first be verified for termination and its semantics summarized; the resulting terminating command can then be treated as part of the body of the surrounding loop. Developing such a compositional treatment systematically, as well as extending the rule to nondeterministic choice and certificate synthesis, is left to future work (Section 10).

Outline. Section 2 introduces probabilistic programs, subdistributions, AST, and distributional invariants. Section 3 recalls the pCFG rule that underpins our metatheory. Section 4 states the new rule and gives intuition. Section 5 proves soundness and relative completeness. The random-walk example appears in Section 7. The FDR comparison and the FLDR and Han–Hoshi case studies appear in Sections 8.1 to 8.3, respectively. We discuss related work in Section 9 and conclude in Section 10.

2 Preliminaries

We fix a finite set Var of program variables and a countable state space Σ ; a state $\sigma \in \Sigma$ maps variables to rational values. Arithmetic and Boolean expressions are interpreted as functions on states. For a Boolean expression b , let $\llbracket b \rrbracket = \{\sigma \mid \sigma \models b\}$ and let $[b]$ denote its Iverson bracket. Updating x in σ to $a(\sigma)$ is written $\sigma[x/a(\sigma)]$.

2.1 Probabilistic programs and subdistributions

We consider the probabilistic guarded-command language

$$C ::= \text{skip} \mid x := a \mid C ; C \mid \{C\} [p] \{C\} \mid \text{if } (b) \{C\} \text{ else } \{C\} \mid \text{while } (b) \{C\},$$

where $p \in [0, 1] \cap \mathbb{Q}$. The language includes probabilistic choice but excludes nondeterministic choice. A *subdistribution* is a function $\mu : \Sigma \rightarrow \mathbb{R}_{\geq 0}$ of weight

$|\mu| = \sum_{\sigma} \mu(\sigma) \leq 1$. We write $\Delta_{\leq 1}\Sigma$ for all subdistributions, $\text{supp}(\mu) = \{\sigma \mid \mu(\sigma) > 0\}$ for the support, and δ_{σ} for the Dirac distribution at σ . Missing mass represents nontermination.

Programs denote linear, ω -continuous post-distribution transformers $\text{sp}[[C]] : \Delta_{\leq 1}\Sigma \rightarrow \Delta_{\leq 1}\Sigma$ that do not increase mass. This is the forward denotational semantics of the probabilistic language: each input subdistribution has one output subdistribution, in the tradition of Kozen [?, ?]. The clauses needed below are

$$\begin{aligned} \text{sp}[[\text{skip}]](\mu) &= \mu, \\ \text{sp}[[x := a]](\mu)(\sigma) &= \sum_{\sigma' : \sigma'[x/a(\sigma')] = \sigma} \mu(\sigma'), \\ \text{sp}[[\{C_1\} [p] \{C_2\}]](\mu) &= \text{sp}[[C_1]](p\mu) + \text{sp}[[C_2]]((1-p)\mu), \\ \text{sp}[[C_1 ; C_2]](\mu) &= \text{sp}[[C_2]](\text{sp}[[C_1]](\mu)), \\ \text{sp}[[\text{if } (b) \{C_1\} \text{ else } \{C_2\}]](\mu) &= \text{sp}[[C_1]]([b]\mu) + \text{sp}[[C_2]]([\neg b]\mu). \end{aligned}$$

For a loop $C = \text{while } (b) \{B\}$, we take a least fixed point over transformers, as in [?]. We order subdistributions and transformers pointwise. Let $\mathcal{H}$ be the pointed ω -cpo of ω -continuous transformers $f : \Delta_{\leq 1}\Sigma \rightarrow \Delta_{\leq 1}\Sigma$ satisfying $|f(\eta)| \leq |\eta|$ for every $\eta \in \Delta_{\leq 1}\Sigma$, with bottom $\perp = \lambda\eta.0$. Define

$$\begin{aligned} \Phi_{b,B}(f)(\eta) &= [\neg b]\eta + f(\text{sp}[[B]]([b]\eta)), \\ \text{sp}[[C]](\mu) &= (\text{lfp } \Phi_{b,B})(\mu) = \sup_{j \in \mathbb{N}} \Phi_{b,B}^j(\perp)(\mu). \end{aligned} \tag{1}$$

The functional $\Phi_{b,B} : \mathcal{H} \rightarrow \mathcal{H}$ is ω -continuous, and every approximant has mass at most $|\mu|$ (see Section A). For $j \geq 1$, $\Phi_{b,B}^j(\perp)(\mu)$ collects the terminating executions with fewer than j completed body iterations. Thus the fixed point ranges over transformers, while all their inputs and outputs remain subdistributions.

Linearity and ω -continuity give $\text{sp}[[C]](\sum_i p_i \delta_{\sigma_i}) = \sum_i p_i \text{sp}[[C]](\delta_{\sigma_i})$ for any finite or countable family with $p_i \geq 0$ and $\sum_i p_i \leq 1$, which lets us reduce one-iteration obligations to Dirac inputs.

Definition 1 (Almost-sure termination). *A program C is almost-surely terminating on $M \subseteq \Delta_{\leq 1}\Sigma$ if $|\text{sp}[[C]](\mu)| = |\mu|$ for every $\mu \in M$.*

Thus AST means that execution loses no probability mass. We do not require positive AST: the expected runtime may be infinite.

2.2 Distributional invariants

Our assertions are sets of distributions rather than predicates over individual states. This aligns the termination argument with existing partial-correctness proofs for sampling algorithms [?].

Definition 2 (Distributional invariant). *Let $C = \text{while } (b) \{B\}$ and $M \subseteq \Delta_{\leq 1}\Sigma$. A set $I \subseteq \Delta_{\leq 1}\Sigma$ is a distributional invariant for C and M if*

$$M \subseteq I \quad \text{and} \quad [\neg b]\mu + \text{sp}[[B]]([b]\mu) \in I \quad \text{for all } \mu \in I.$$

We abbreviate $\text{supp}(I) = \bigcup_{\mu \in I} \text{supp}(\mu)$.

The invariant can simultaneously carry state-space bounds and genuinely distributional information such as equal probability of states in a row of a decision tree. The AST rule uses the support information, while retaining the stronger invariant permits direct composition with partial correctness.

2.3 Supermartingales, variants, and sublevel sets

A function $v : \text{supp}(I) \rightarrow \mathbb{R}_{\geq 0}$ is a one-iteration supermartingale for loop body B if, for every enabled state σ ,

$$\sum_{\sigma' \in \Sigma} \text{sp}[[B]](\delta_\sigma)(\sigma') v(\sigma') \leq v(\sigma).$$

For $r \in \mathbb{R}_{\geq 0}$, its sublevel set is $v_{\leq r} = \{\sigma \in \text{supp}(I) \mid v(\sigma) \leq r\}$. A variant $u : \text{supp}(I) \rightarrow \mathbb{N}$ need not decrease in expectation or on every outcome. We only require a uniformly positive chance to move to a smaller value. The role of v is to ensure that executions cannot escape all regions in which u has a finite upper bound.

This division of labour matters for rejection samplers. A rejected run may reset u to a larger value; AST follows because progress remains possible with nonzero probability while v controls the size of the relevant search space.

3 AST Certificates for Probabilistic Control-Flow Graphs

Our program-level rule builds on the sound and relatively complete pCFG rule of Majumdar and Sathiyararayanan [?]. Their framework includes bounded demonic nondeterminism and requires AST under every scheduler. We state the specialization to pCFGs without nondeterministic choice, matching the language of Section 2, before relating its location-dependent certificates to our distribution-transformer formulation.

A probabilistic control-flow graph (pCFG) has configurations (ℓ, σ) consisting of a control location ℓ and a program state σ . Its locations are partitioned into assignment locations L_A , conditional locations L_C , and probabilistic locations L_P . Assignment and conditional configurations have a unique enabled successor. At a probabilistic location, an outgoing edge is selected according to its fixed probability. The distinguished terminal location ℓ_{out} is absorbing.

For a configuration $\gamma = (\ell, \sigma)$, let $\text{Succ}(\gamma)$ denote the set of its successor configurations. If $\ell \in L_P$, let $\text{Pr}(\gamma, \gamma')$ denote the probability of moving from γ to $\gamma' \in \text{Succ}(\gamma)$. An invariant Inv is *inductive* for a set E of initial states if

$$\{(\ell_{\text{init}}, \sigma) \mid \sigma \in E\} \subseteq \text{Inv}$$

and Inv is closed under all positive-probability transitions.

The rule uses two certificates. A non-negative supermartingale V controls the probability of reaching configurations arbitrarily far from termination. A natural-valued variant U provides local progress towards termination. For $r \in \mathbb{R}_{\geq 0}$, define the V -sublevel set

$$V_{\leq r} = \{\gamma \in \text{Inv} \mid V(\gamma) \leq r\}.$$

Theorem 1 (pCFG AST rule, adapted from [?]). *Let $\mathcal{G}$ be a pCFG with initial location ℓ_{init} and terminal location ℓ_{out} , and let E be a set of initial states. The pCFG $\mathcal{G}$ is almost-surely terminating from E if there exist the following objects:*

1. *An inductive invariant Inv containing $(\ell_{\text{init}}, \sigma)$ for every $\sigma \in E$.*
2. *A function $V: \text{Inv} \rightarrow \mathbb{R}_{\geq 0}$ satisfying:*

(a)

$$V(\ell, \sigma) = 0 \iff \ell = \ell_{\text{out}}.$$

- (b) *For every $\gamma = (\ell, \sigma) \in \text{Inv}$ with $\ell \in L_A \cup L_C$ and unique successor $\gamma' \in \text{Succ}(\gamma)$,*

$$V(\gamma') \leq V(\gamma).$$

- (c) *For every $\gamma = (\ell, \sigma) \in \text{Inv}$ with $\ell \in L_P$,*

$$\sum_{\gamma' \in \text{Succ}(\gamma)} \Pr(\gamma, \gamma') V(\gamma') \leq V(\gamma).$$

3. *A function $U: \text{Inv} \rightarrow \mathbb{N}$ satisfying:*

(a)

$$U(\ell, \sigma) = 0 \iff \ell = \ell_{\text{out}}.$$

- (b) *For every $\gamma = (\ell, \sigma) \in \text{Inv}$ with $\ell \in L_A \cup L_C$ and unique successor $\gamma' \in \text{Succ}(\gamma)$,*

$$U(\gamma') < U(\gamma).$$

- (c) *For every $r \in \mathbb{R}_{\geq 0}$:*

- i. *the set $\{U(\gamma) \mid \gamma \in V_{\leq r}\}$ is bounded; and*
- ii. *there exists $\varepsilon_r > 0$ such that, for every $\gamma = (\ell, \sigma) \in V_{\leq r}$ with $\ell \in L_P$,*

$$\sum_{\substack{\gamma' \in \text{Succ}(\gamma) \\ U(\gamma') < U(\gamma)}} \Pr(\gamma, \gamma') \geq \varepsilon_r.$$

Moreover, the rule is relatively complete with respect to the underlying first-order theory used to express the invariant and the certificates.

The soundness argument has two complementary layers. Fix a sublevel set $V_{\leq r}$. Since U is natural-valued and bounded on this set, it ranges over a finite interval. At deterministic locations it strictly decreases, while at probabilistic locations it decreases with probability at least ε_r . Consequently, a run that remains inside $V_{\leq r}$ cannot avoid termination with positive probability.

It remains to account for runs that leave every fixed sublevel set. The supermartingale conditions ensure that V does not increase in expectation. Standard supermartingale arguments therefore make indefinite escape through increasingly large sublevel sets a probability-zero event. Combining both layers yields almost-sure termination. Unlike proof rules based on a globally bounded variant, the sublevel-set formulation also handles null-recurrent programs, such as the symmetric one-dimensional random walk.

Although semantically powerful, the pCFG rule is syntactically fine-grained. Suppose that one source-level loop iteration consists of a guard, two assignments, a probabilistic choice, and a conditional. Its pCFG translation introduces a separate control location for each of these operations. A proof must therefore define U and V at every intermediate location. Location-dependent constants, scaling factors, and offsets may be needed solely to ensure the required inequalities along the internal transitions of one source-level iteration.

Our rule retains the same two-layer termination argument but moves it to program level. Instead of assigning certificates to intermediate control locations, it contracts each complete execution of a loop-free body into one distribution-transformer step. The certificates can therefore express progress over complete loop iterations, matching the abstraction at which sampling algorithms are naturally understood.

Translation assumption. The standard structural translation $\mathcal{T}[C]$ maps program syntax to a pCFG. Assignments become update edges, conditionals become guarded edges, and probabilistic choices become probabilistic locations. Sequential composition connects the exit of one component to the entry of the next, while a while-loop connects the body exit back to the guard.

For a loop-free body B , paths through $\mathcal{T}[B]$ from its entry location ℓ_{in}^B to its exit location ℓ_{out}^B correspond exactly to the outcomes of its distribution-transformer semantics:

$$\text{sp}[[B]](\delta_\sigma)(\sigma') = \Pr(\text{paths in } \mathcal{T}[B] \text{ from } (\ell_{\text{in}}^B, \sigma) \text{ to } (\ell_{\text{out}}^B, \sigma')).$$

Consequently, a program C is AST on a set M of initial subdistributions exactly when $\mathcal{T}[C]$ reaches its terminal location almost surely from every state in $\text{supp}(M)$. A structural proof of this correspondence is summarized in Section A. This correspondence provides the bridge used in Section 5 to transfer soundness and relative completeness between the pCFG rule and our program-level rule.

4 A Program-Level Rule for AST

Consider a program $C = \text{while } (b) \{B\}$ whose body B is loop-free. Let M be a set of initial subdistributions. We now state the main contribution.

Theorem 2 (Program-level AST rule). *The loop C is AST on M if there exist:*

- (1) *a distributional invariant I for C and M ;*

- (2) a function $v : \text{supp}(I) \rightarrow \mathbb{R}_{\geq 0}$ such that
- (a) $v(\sigma) = 0$ iff $\sigma \in \text{supp}(I) \cap \llbracket \neg b \rrbracket$;
 - (b) for every $\sigma \in \text{supp}(I) \cap \llbracket b \rrbracket$,

$$\sum_{\sigma'} \text{sp}\llbracket B \rrbracket(\delta_\sigma)(\sigma') v(\sigma') \leq v(\sigma);$$

- (3) a function $u : \text{supp}(I) \rightarrow \mathbb{N}$ such that
- (a) $u(\sigma) = 0$ for every $\sigma \in \text{supp}(I) \cap \llbracket \neg b \rrbracket$;
 - (b) there exists $\varepsilon > 0$ for which, for every enabled state $\sigma \in \text{supp}(I) \cap \llbracket b \rrbracket$;

$$\sum_{\sigma' : u(\sigma') < u(\sigma)} \text{sp}\llbracket B \rrbracket(\delta_\sigma)(\sigma') \geq \varepsilon; \quad (\text{Decrease})$$

- (c) for every $r \geq 0$, the set $\{u(\sigma) \mid \sigma \in \text{supp}(I), v(\sigma) \leq r\}$ is bounded.

The conditions deliberately do not require u to decrease in expectation. It may increase sharply on some outcomes, as happens when a rejection sampler restarts. The rule asks only for a fixed positive mass of outcomes that reduce u . Nor must u be globally bounded. Its boundedness is local to sublevel sets of v , allowing null-recurrent computations.

Soundness proof intuition. Fix a sublevel set $v_{\leq r}$ and let K_r bound u there. From every nonterminal state in this set, a complete loop iteration has probability at least ε of decreasing u . Hence there is probability at least ε^{K_r} of following enough decreasing iterations to reach zero, provided the run stays in the set. Repeating this argument rules out nontermination confined to $v_{\leq r}$. Meanwhile, the supermartingale property bounds the probability of reaching arbitrarily large values of v ; non-negative supermartingale convergence rules out escaping every finite sublevel set with positive probability. Thus all nonterminating behavior has measure zero.

Iteration-level abstraction. The essential difference to Theorem 1 is the granularity of the decrease condition. For a body containing m internal control-flow steps, the pCFG rule needs a value at each intermediate location and a descent-compatible branch at each probabilistic node. The new rule evaluates the final post-distribution once. Assignments, guards, and choices internal to B are hidden behind $\text{sp}\llbracket B \rrbracket$. This has three practical consequences:

1. certificates are functions of program states, not location–state pairs;
2. an existing distributional invariant can be reused directly; and
3. algebraic simplification of $\text{sp}\llbracket B \rrbracket(\delta_\sigma)$ exposes exactly the outcomes relevant to progress.

Uniform versus sublevel-dependent progress. The pCFG rule permits an ε_r for each sublevel set. In our loop-free bodies, fixed probabilistic choices and bounded path length give a positive lower bound on the probability of any feasible body path. Thus the existence of a decreasing path yields a global progress bound; all three sampler proofs use $\varepsilon = 1/2$. A more general statement can replace ε by ε_r throughout without changing the proof architecture.

Modularity. Initial deterministic code preceding the loop is handled by transforming M into the set of loop-entry distributions. For a sampler with preprocessing, we regard the resulting finite data structure as read-only parameters of the sampling loop. The preprocessing itself terminates by ordinary bounded-loop reasoning. This isolates the probabilistic AST argument to the loop where it is needed.

5 Soundness and Relative Completeness

We connect Theorem 2 to the pCFG rule through the translation outlined in Section 3. The proofs also explain why contracting a loop body preserves the two certificates.

5.1 From edge-level to iteration-level reasoning

Let B be loop-free and let $\mathcal{T}[B]$ have entry ℓ_{in} and exit ℓ_{out} . Because B is loop-free, every path from entry to exit is finite and there is a maximum path length L_B .

Lemma 1 (Supermartingale contraction). *Let V satisfy the pCFG supermartingale conditions on $\mathcal{T}[B]$. Then for every reachable (ℓ, σ) ,*

$$V(\ell, \sigma) \geq \sum_{\sigma'} \Pr(\ell, \sigma \rightsquigarrow \ell_{\text{out}}, \sigma') V(\ell_{\text{out}}, \sigma').$$

Proof. Induct backwards over the acyclic control-flow structure of B . At an assignment or guard location, apply monotonicity along the unique enabled successor. At a probabilistic location, apply the local expectation inequality and the induction hypothesis to every successor. The path probabilities partition according to the first edge. $\square$

Lemma 2 (Variant path). *Let U satisfy the pCFG variant conditions on $\mathcal{T}[B]$. From every reachable entry configuration there is a positive-probability path to an exit configuration with a smaller U -value.*

Proof. At deterministic locations choose the unique successor, which lowers U . At probabilistic locations choose a positive-probability successor lowering U . Acyclicity ensures that this construction reaches the body exit after finitely many choices. $\square$

5.2 Soundness

Theorem 3 (Soundness). *The program-level rule in Theorem 2 is sound.*

Proof (Proof sketch). Assume certificates I, v, u and translate C to $G = \mathcal{T}[C]$. Let Inv contain all configurations reachable from $\text{supp}(M)$. Distributional inductiveness and the translation correspondence ensure that every state at the loop guard belongs to $\text{supp}(I)$.

We lift v to a pCFG supermartingale V . At the loop guard set $V(\ell_g, \sigma) = v(\sigma) + 1$, and set $V = 0$ at the program exit. At an internal body location, define V as the expected value at the end of the remaining body execution. This satisfies equality at probabilistic locations and along all internal deterministic steps. At the back edge, the source-level supermartingale inequality establishes non-increase. Lemma 1 formalizes this construction.

For the variant, reserve enough numerical room for internal control flow. At the guard put $U(\ell_g, \sigma) = L_B u(\sigma) + 1$ and put $U = 0$ at the program exit. Define values backwards inside the body: at a deterministic location use the successor value plus one; at a probabilistic location use one plus the minimum successor value. Thus every deterministic edge decreases U , and some probabilistic successor decreases it.

The source-level decrease condition says that, with probability at least ε , a body execution ends in σ' with $u(\sigma') < u(\sigma)$. Since its path has length at most L_B , scaling by L_B ensures that at least one such path descends through the internal pCFG values and returns to a smaller guard value. Fixed program probabilities provide the required positive lower bound at internal probabilistic locations.

Finally, if $v(\sigma) \leq r$, source-level boundedness bounds $u(\sigma)$; multiplication by L_B and addition of at most L_B internal steps preserves boundedness of U . The lifted certificates satisfy Theorem 1, so G reaches its exit almost surely. AST preservation by the translation yields $|\text{sp}[[C]](\mu)| = |\mu|$ for every $\mu \in M$. $\square$

5.3 Relative completeness

Theorem 4 (Relative completeness). *For AST loops $C = \text{while}(b) \{B\}$ with loop-free body, the program-level rule is relatively complete with respect to the first-order arithmetic theory in which its assertions and certificates are expressed.*

Proof (Proof sketch). Suppose C is AST on M . Its translated pCFG G is AST from $\text{supp}(M)$. Relative completeness of Theorem 1 yields an inductive invariant Inv and pCFG certificates V, U .

Project the guard-location invariant to distributions:

$$I = \{\mu \in \Delta_{\leq 1} \Sigma \mid \text{supp}(\mu) \subseteq \{\sigma \mid (\ell_g, \sigma) \in Inv\}\}.$$

Operational/denotational correspondence and inductiveness of Inv make I a distributional invariant. Set $v(\sigma) = V(\ell_g, \sigma)$ and $u(\sigma) = U(\ell_g, \sigma)$. The expected non-increase of v after a complete iteration follows from Lemma 1 and the equality between path probabilities and $\text{sp}[[B]](\delta_\sigma)$.

By Lemma 2, some body path reaches a lower guard-level U -value. Inside a fixed V -sublevel set, the pCFG rule supplies positive lower bounds for all

probabilistic descent choices. A body path has at most L_B edges, so the product of finitely many such bounds is positive; taking the minimum over the bounded certificate range yields the source-level decrease bound. Restricting the pCFG boundedness condition to guard configurations directly bounds u on every v -sublevel set.

The projected objects are definable whenever the original pCFG certificates and reachability assertions are definable in the underlying theory. They satisfy Theorem 2. $\square$

The two translations show that no semantic proving power is lost for the considered program class. The benefit is representational: users state certificates once per source state and reason about complete iterations.

6 Template-Based Certificate Synthesis

We instantiate the program-level rule with an SMT-based search for two certificates over loop-head states. The current prototype takes a supplied inductive state predicate and a manually encoded one-iteration semantics. It synthesizes the coefficients of the certificates; invariant generation and automatic translation of source programs are outside its current scope.

6.1 Guarded Affine Templates and Constraints

Consider a loop with guard b over integer-valued variables $\mathbf{x} = (x_1, \dots, x_d)$. Let S be a supplied predicate containing the initial states and preserved by every enabled body execution. For the presented examples, the body has two equiprobable outcomes represented by total, piecewise affine updates T_0, T_1 . Both updates describe an entire body execution, including its deterministic branches.

We use independent templates

$$V_{\mathbf{a}}(\mathbf{x}) = \begin{cases} a_0 + \sum_{i=1}^d a_i x_i, & b(\mathbf{x}), \\ 0, & \neg b(\mathbf{x}), \end{cases} \quad U_{\mathbf{c}}(\mathbf{x}) = \begin{cases} c_0 + \sum_{i=1}^d c_i x_i, & b(\mathbf{x}), \\ 0, & \neg b(\mathbf{x}). \end{cases}$$

Both functions have integer coefficients between -2 and 2 . The prototype prefers a small sum of absolute coefficient values. This preference guides the search and has no role in certificate validity.

For every $\mathbf{x}$ satisfying $S \wedge b$, we impose

$$V(\mathbf{x}) \geq 1, \quad U(\mathbf{x}) \geq 1, \quad (2)$$

$$V(T_0(\mathbf{x})) + V(T_1(\mathbf{x})) \leq 2V(\mathbf{x}), \quad (3)$$

$$U(T_0(\mathbf{x})) \leq U(\mathbf{x}) - 1 \quad \vee \quad U(T_1(\mathbf{x})) \leq U(\mathbf{x}) - 1, \quad (4)$$

$$U(\mathbf{x}) \leq V(\mathbf{x}). \quad (5)$$

The last condition ensures that U is bounded on sublevel sets of V . It is stronger than the corresponding premise of the original rule. Likewise, integer coefficients and $V \geq 1$ restrict the search space beyond the requirements of that rule.

Proposition 1 (Soundness of the synthesis constraints). *Assume that S contains the initial states and is preserved by both body outcomes. Every template instantiation satisfying (1)–(4) for all states in $S \wedge b$ is an AST certificate with progress bound $\varepsilon = 1/2$.*

Proof. The templates vanish off the guard. On enabled states, V is positive and U is a positive integer. Equation (2) is the one-iteration supermartingale inequality. Equation (3) ensures a strict decrease of U on an outcome of probability $1/2$. Finally, $U \leq V$ on S , including terminal states, implies that $U \leq \lfloor r \rfloor$ whenever $V \leq r$. The set of subdistributions supported in S is a distributional invariant for the loop and the allowed inputs. Hence Theorem 2 applies. $\square$

6.2 Counterexample-Guided Search

The learner instantiates the constraints on a finite set of enabled states and selects candidate coefficients. The verifier fixes these coefficients and asks whether there exists any integer state satisfying $S \wedge b$ that violates a constraint. A satisfying state is added to the learner’s sample set; an unsatisfiable query establishes all constraints throughout the invariant domain. Unknown results and resource limits are treated as inconclusive.

With the states fixed, the learner’s constraints are linear in the coefficients. With the coefficients fixed, the verifier’s constraints are Boolean combinations of linear arithmetic over states. Thus the procedure avoids solving the original existential-universal formula directly. The final verification places no bound on the program variables. Initialization and preservation of S are checked independently.

For a fixed finite coefficient domain, every counterexample excludes the current candidate. In the absence of resource limits and with terminating exact solvers, this gives a terminating search that is complete only for the chosen finite template space. It does not establish completeness for affine certificates with unrestricted coefficients or for AST programs.

6.3 Two Worked Synthesis Examples

Unbounded symmetric random walk. Consider the loop

$$\text{while } (x > 0) \{x := x - 1 \text{ } [1/2] \text{ } x := x + 1\}$$

over integers with $S \equiv x \geq 0$. The templates are $V(x) = ax + b$ and $U(x) = cx + d$ on enabled states, and zero at $x = 0$. In the recorded run, the learner starts from $x = 1$ and first proposes $V = U = 1$ on enabled states. The verifier returns $x = 2$: both successors remain enabled and keep $U = 1$, violating progress. The next candidate is $V(x) = U(x) = x$, which is verified for every $x > 0$:

$$\frac{1}{2}V(x-1) + \frac{1}{2}V(x+1) = x = V(x), \quad U(x-1) = U(x) - 1, \quad U = V.$$

Both functions are nonnegative on S and vanish at zero. The invariant is preserved because $x \geq 1$ before either update. This proves AST from every nonnegative integer initial state.

This example separates AST synthesis from finite-expected-runtime arguments. For $0 < x < N$, the expected time to hit either 0 or N is $x(N - x)$, which lower-bounds the expected time to hit 0. Letting N grow shows that the latter expectation is infinite for $x > 0$. The synthesized V is a martingale, not a certificate of uniform expected decrease. The separate probabilistic progress condition still holds.

Fast Dice Roller with symbolic range. Use the FDR loop of the case study, with initial values $v = 1, c = 0$ and symbolic integer parameter $n \geq 1$. Its invariant domain is

$$S \equiv n \geq 1 \wedge 1 \leq v < 2n \wedge 0 \leq c < v \wedge c < n.$$

For bit $\beta \in \{0, 1\}$, the complete body update is

$$T_\beta(n, v, c) = \begin{cases} (n, 2v - n, 2c + \beta - n), & 2c + \beta \geq n, \\ (n, 2v, 2c + \beta), & 2c + \beta < n. \end{cases}$$

Independent affine templates over (n, v, c) yield the verified pair

$$V(n, v, c) = \begin{cases} n, & v < n, \\ 0, & v \geq n, \end{cases} \quad U(n, v, c) = \begin{cases} n - v + c, & v < n, \\ 0, & v \geq n. \end{cases}$$

The shared engine finds this pair after four candidates in the recorded run. Both certificates are verified for all integer states in S , with no fixed upper bound on n .

At an enabled state, let $d = v - c$. Then $1 \leq d \leq n - 1$, so $1 \leq U = n - d \leq n - 1 < V$. Since n is unchanged, either successor has V equal to n or zero, proving the supermartingale condition. On the zero-bit outcome, the difference becomes $d' = 2d$: subtracting n from both variables leaves their difference unchanged. If the successor remains enabled, $U' = n - 2d \leq U - 1$; otherwise $U' = 0 < U$. Thus progress holds with probability at least $1/2$, and $U \leq V$ establishes sublevel boundedness.

The affine variant therefore suffices for qualitative termination; the logarithmic variant used in the manual proof is not required for this purpose. FDR is nevertheless a baseline: on the one-bit outcome, $d' = 2d - 1 \geq d$, or execution terminates, so $U' \leq U$. Consequently $\mathbb{E}[U'] \leq U - 1/2$, and this particular U is itself a ranking supermartingale. Together, the two examples demonstrate symbolic-parameter synthesis and AST with infinite expected runtime; they do not yet establish an advantage over existing synthesis tools.

7 The One-Dimensional Random Walk

Consider the symmetric random walk

$$x := k; \quad \text{while } (x > 0) \{ \{x := x - 1\} [1/2] \{x := x + 1\} \}, \quad k \in \mathbb{N}_{>0}.$$

This loop terminates almost surely at $x = 0$, although its expected termination time is infinite. It is therefore a compact illustration of why the supermartingale and variant play different roles.

Let $M = \{\delta_\sigma \mid \sigma(x) = k, k > 0\}$ and $I = \{\mu \in \Delta_{\leq 1}\Sigma \mid \text{supp}(\mu) \subseteq \{\sigma \mid \sigma(x) \geq 0\}\}$. The set I is distributionally inductive. Choose

$$v(\sigma) = u(\sigma) = \max\{\sigma(x), 0\}.$$

For every enabled state,

$$\sum_{\sigma'} \text{sp}[[B]](\delta_\sigma)(\sigma') v(\sigma') = \frac{1}{2}(x-1) + \frac{1}{2}(x+1) = x = v(\sigma),$$

so v is in fact a martingale. With probability $1/2$, the left branch is taken and u decreases by one; hence the decrease obligation holds with $\varepsilon = 1/2$. Finally, on $v \leq r$ we have $u \leq r$. Theorem 2 proves AST.

The same mathematical idea can be expressed at pCFG level, but the certificate becomes location-dependent. A loop iteration contains the guard, a probabilistic choice, and an assignment. A pCFG variant therefore uses a scaling factor (three in a direct translation) and different additive offsets at the intermediate locations to force strict descent across every deterministic edge. The supermartingale similarly anticipates the next assignment at branch locations. None of this bookkeeping represents probabilistic insight; it is introduced by the operational granularity. The program rule preserves the natural proof: “distance x is a martingale and decreases with probability one half.”

A walled walk with $0 \leq x \leq W$ admits an even older bounded-variant argument [?]. Removing the wall destroys global boundedness but not AST. Sublevel-set boundedness is precisely what recovers the unbounded example and previews the sampler proofs below.

The same construction extends to the standard two-dimensional walk when termination is phrased as hitting an appropriate absorbing boundary and a suitable radial or coordinate certificate is available. We keep the one-dimensional version because it isolates the proof rule without geometric side conditions; the substantial case studies concern sampling algorithms.

8 Case Studies: Exact Sampling from Fair Bits

8.1 Fast Dice Roller: the same proof at two levels

The Fast Dice Roller (FDR) samples uniformly from $\{0, \dots, n-1\}$ using fair coin flips [?]. We use the following program-level presentation:

```

v := 1; c := 0;
while (v < n) do
  v := 2v;
  c := 2c [1/2] c := 2c + 1;
  if (c ≥ n) then v := v - n; c := c - n else skip
end
    
```

On termination, c is the sampled outcome. Existing distributional-invariant reasoning proves that the terminating output is uniform [?]; AST completes total correctness. The distributional invariant taken from [?] states that at the loop head, the reachable states satisfy

$$0 \leq c < \min(v, n), \quad 0 < v < 2n, \quad n > 0. \quad (6)$$

The distributional invariant additionally states that equal admissible c -values carry equal mass, however only (5) is needed for our termination proof.

Let B_{FDR} denote the body. For a Dirac input δ_σ , its post-distribution is

$$\begin{aligned} \text{sp}[B_{\text{FDR}}](\delta_\sigma) &= \frac{1}{2}[c < n](\delta_{\sigma[v/2v, c/2c]} + \delta_{\sigma[v/2v, c/2c+1]}) \\ &\quad + \frac{1}{2}[c \geq 0](\delta_{\sigma[v/2v-n, c/2c-n]} + \delta_{\sigma[v/2v-n, c/2c+1-n]}), \end{aligned} \quad (7)$$

where brackets are evaluated after the corresponding updates. This compact expression is the only operational calculation required by the program-level proof.

Theorem 5 (AST of FDR). *For every $n \in \mathbb{N}_{>0}$, FDR terminates almost surely.*

Proof. Use the existing distributional invariant and define

$$v(\sigma) = \begin{cases} 0 & v \geq n, \\ n & v < n, \end{cases} \quad u(\sigma) = \begin{cases} 0 & v \geq n, \\ \left\lceil \log_2 \left(\frac{n}{v-c} \right) \right\rceil & v < n. \end{cases}$$

The supermartingale obligation is immediate: n is unchanged and every successor is assigned either n or 0. For progress, inspect the outcome generated by the left fair branch. Before possible subtraction of n , both v and c are doubled; after subtraction their difference remains $2(v-c)$. Hence in either case the denominator of $n/(v-c)$ doubles, so u strictly decreases. The left bit has probability $1/2$, giving $\varepsilon = 1/2$.

For a sublevel set with $r < n$, only terminal states occur. If $r \geq n$, the invariant bounds $n \leq r$, $0 < v < 2n$, and $0 \leq c < v$, so $v - c \geq 1$ and $u \leq \lceil \log_2 r \rceil$. All obligations of Theorem 2 hold. $\square$

Comparison with the pCFG proof. The pCFG rule of Majumdar and Sathiyarayanarayana is applicable to FDR and establishes the same AST statement as Theorem 5. Unlike the program-level proof, however, it reasons about individual configurations (ℓ, σ) of the graph in Figure 1. Thus, the initialisation, loop guard, doubling of v , fair-bit choice, conditional, and the two subtraction assignments all become separate proof obligations.

For completeness, we spell out the corresponding pCFG certificate. Write Inv_ℓ for the states admitted at location ℓ . The invariant is obtained by trans-

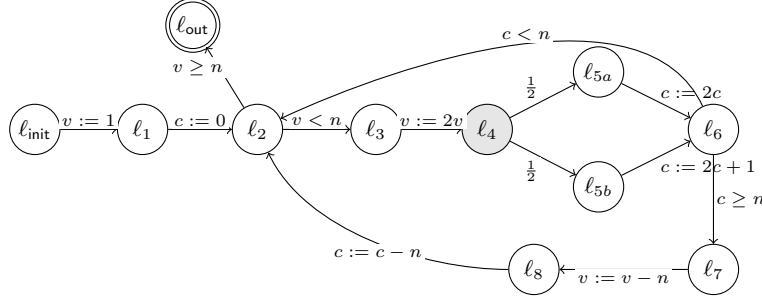


Fig. 1. The pCFG obtained from FDR. The locations $\ell_2, \dots, \ell_8$ expose the loop guard, the update $v := 2v$, the fair-bit choice, the bit-dependent updates of c , and the rejection branch.

porting the support information (5) through the intermediate assignments:

$$\begin{aligned}
 Inv_{\ell_{\text{init}}} &:= \{\sigma \mid \sigma(n) \in \mathbb{N}_{>0}\}, \\
 Inv_{\ell_1} &:= \{\sigma \mid \sigma(n) \in \mathbb{N}_{>0} \wedge \sigma(v) = 1\}, \\
 Inv_{\ell_2} &:= \{\sigma \mid \sigma(n) \in \mathbb{N}_{>0} \wedge 1 \leq \sigma(v) < 2\sigma(n) \wedge 0 \leq \sigma(c) < \min\{\sigma(v), \sigma(n)\}\}, \\
 Inv_{\ell_3} &:= \{\sigma \in Inv_{\ell_2} \mid \sigma(v) < \sigma(n)\}, \\
 Inv_{\ell_4} &:= \{\sigma \mid \sigma(n) \in \mathbb{N}_{>0} \wedge 2 \leq \sigma(v) \leq 2\sigma(n) - 2 \wedge 0 \leq \sigma(c) < \frac{\sigma(v)}{2}\}, \\
 Inv_{\ell_{5a}} &:= Inv_{\ell_4}, \quad Inv_{\ell_{5b}} := Inv_{\ell_4}, \\
 Inv_{\ell_6} &:= \{\sigma \mid \sigma(n) \in \mathbb{N}_{>0} \wedge 2 \leq \sigma(v) \leq 2\sigma(n) - 2 \wedge 0 \leq \sigma(c) < \sigma(v)\}, \\
 Inv_{\ell_7} &:= \{\sigma \in Inv_{\ell_6} \mid \sigma(c) \geq \sigma(n)\}, \\
 Inv_{\ell_8} &:= \{\sigma \mid \sigma(n) \in \mathbb{N}_{>0} \wedge 1 \leq \sigma(v) \leq \sigma(n) - 2 \wedge \sigma(n) \leq \sigma(c) < \sigma(v) + \sigma(n)\}, \\
 Inv_{\ell_{\text{out}}} &:= \{\sigma \in Inv_{\ell_2} \mid \sigma(v) \geq \sigma(n)\}.
 \end{aligned}$$

Hence

$$Inv := \bigcup_{\ell \in \{\ell_{\text{init}}, \ell_1, \dots, \ell_8, \ell_{\text{out}}\}} (\{\ell\} \times Inv_{\ell})$$

is an inductive pCFG invariant. The loop-head component Inv_{ℓ_2} is precisely the support part of the distributional invariant reused in the program-level proof; the other components are location-specific consequences of it.

The supermartingale is simple but already location-dependent through the terminal location:

$$V(\ell, \sigma) := \begin{cases} 0, & \ell = \ell_{\text{out}}, \\ \sigma(n), & \ell \neq \ell_{\text{out}}. \end{cases}$$

Since n is never modified, every nonterminal deterministic successor has the same V -value as its predecessor, while a transition to ℓ_{out} decreases it to 0. At the sole probabilistic location ℓ_4 , both successors retain the value $\sigma(n)$. Thus the supermartingale obligations are immediate. Moreover, every sublevel set $V_{\leq r}$

fixes $n \leq r$; together with the bounds in *Inv* this makes all non-initial parts of the sublevel set finite.

The pCFG variant must instead account for the exact point within an iteration. One suitable choice is the following location-specific refinement of the iteration-level distance $\lceil \log_2(n/(v-c)) \rceil$:

$$U(\ell, \sigma) := \begin{cases} 7 \lceil \log_2(\sigma(n)) \rceil + 3, & \ell = \ell_{\text{init}}, \\ 7 \lceil \log_2(\sigma(n)) \rceil + 2, & \ell = \ell_1, \\ 7 \left\lceil \log_2 \left(\frac{\sigma(n)}{\sigma(v) - \sigma(c)} \right) \right\rceil + 1, & \ell = \ell_2, \\ 7 \left\lceil \log_2 \left(\frac{\sigma(n)}{\sigma(v) - \sigma(c)} \right) \right\rceil, & \ell = \ell_3, \\ 7 \left\lceil \log_2 \left(\frac{\sigma(n)}{\frac{\sigma(v)}{2} - \sigma(c)} \right) \right\rceil - 1, & \ell = \ell_4, \\ 7 \left\lceil \log_2 \left(\frac{\sigma(n)}{\sigma(v) - 2\sigma(c)} \right) \right\rceil + 5, & \ell = \ell_{5a}, \\ 7 \left\lceil \log_2 \left(\frac{\sigma(n)}{\sigma(v) - (2\sigma(c) + 1)} \right) \right\rceil + 5, & \ell = \ell_{5b}, \\ 7 \left\lceil \log_2 \left(\frac{\sigma(n)}{\sigma(v) - \sigma(c)} \right) \right\rceil + 4, & \ell = \ell_6, \\ 7 \left\lceil \log_2 \left(\frac{\sigma(n)}{\sigma(v) - \sigma(c)} \right) \right\rceil + 3, & \ell = \ell_7, \\ 7 \left\lceil \log_2 \left(\frac{\sigma(n)}{\sigma(v) + \sigma(n) - \sigma(c)} \right) \right\rceil + 2, & \ell = \ell_8, \\ 0, & \ell = \ell_{\text{out}}. \end{cases}$$

The factor 7 bounds the number of pCFG transitions needed for one source-level loop iteration. The additive offsets ensure strict descent on transitions that merely change the control location. The expressions at $\ell_4, \ell_{5a}, \ell_{5b}$ additionally compensate for the fact that one of the two updates of the iteration has already been executed. For example, at ℓ_4 the current value of v has already been doubled whereas c still has its old value, explaining the denominator $\sigma(v)/2 - \sigma(c)$. At ℓ_{5a} and ℓ_{5b} , the expressions anticipate the respective updates of c , which are executed on the outgoing edges. The deterministic decrease conditions can consequently be discharged by checking the edges of Figure 1; the guard edge to ℓ_{out} decreases to 0, and the offsets enforce strict decrease on the remaining deterministic edges. At ℓ_4 , the ℓ_{5a} successor—corresponding to the fair-bit outcome 0—has a strictly smaller variant. This successor has probability $\frac{1}{2}$. Semantically, this is the familiar iteration-level observation that choosing the zero bit doubles $v - c$; the rejection updates subtract n from both variables and hence preserve that difference. In the pCFG proof this single observation is distributed over the locations $\ell_4, \ell_{5a}, \ell_6, \ell_7, \ell_8$.

Finally, U is bounded in every sublevel set $V_{\leq r}$. Indeed, $V \leq r$ implies $\sigma(n) \leq r$, and the invariant bounds all relevant values of v and c ; the two initialization locations are harmless because their variant values depend only on n . Thus the pCFG rule applies with $\varepsilon_r = \frac{1}{2}$ for every r .

In comparison, the program-level proof uses one support invariant, the two state functions displayed in Theorem 5, and the single body calculation (6). It also directly reuses the invariant required for the uniform-output proof. Both proof routes have the same proving power, but the program-level rule expresses the algorithmic fact—progress over a complete loop iteration—without introducing locations, offsets, and intermediate state expressions solely for control-flow bookkeeping.

8.2 Fast Loaded Dice Roller: rejection without graph bookkeeping

The Fast Loaded Dice Roller (FLDR) samples outcome i with probability a_i/m , where $a_1, \dots, a_n$ are positive integers and $m = \sum_{i=1}^n a_i$ [?]. Let $k = \lceil \log_2 m \rceil$. Preprocessing adds a rejecting outcome $n+1$ of weight $2^k - m$ and builds a compact representation (H, h) of a binary decision tree. Sampling traverses the tree using fair bits; reaching $n+1$ resets the traversal to the root.

Abstracting the terminating preprocessing, the sampling loop is:

```

while (d <  $\phi_h(c)$ ) do
  c := c + 1;
  d := 2d [1/2] d := 2d + 1;
  if (H[d, c] = n + 1) then c := 0; d := 0 else skip
end
    
```

Here c is the current depth and d the node index; $\phi_h(c)$ identifies the first leaf position in row c . The preprocessing guarantees two facts used below: the tree height is at most k , and no internal node has both children equal to the rejecting leaf. Equivalently, from every enabled state at least one fair-bit successor avoids reset and advances one level towards a valid leaf.

The distributional invariant from the partial-correctness proof bounds reachable states by

$$0 \leq c \leq k, \quad 0 \leq d \leq n, \quad (8)$$

and additionally records equal-mass and output-probability constraints. Again, the latter are retained for total correctness but are not consumed by AST.

Theorem 6 (AST of FLDR). *For every finite vector of positive integer weights, the FLDR sampling loop terminates almost surely.*

Proof. Let b be the loop guard. Define

$$v(\sigma) = \begin{cases} 0 & \sigma \not\models b, \\ m & \sigma \models b, \end{cases} \quad u(\sigma) = \begin{cases} 0 & \sigma \not\models b, \\ k - c & \sigma \models b. \end{cases}$$

Neither the weights nor m are modified. Every post-state therefore has v equal to m if the loop remains enabled and 0 otherwise, proving the supermartingale inequality.

Consider an enabled node. Each fair-bit outcome first increments c . If it is not the rejecting leaf, no reset occurs and u decreases by one. The preprocessing property excludes the case in which both children reject. Therefore at least one of the two outcomes has smaller u , and its probability is $1/2$. The decrease condition holds with $\varepsilon = 1/2$ even though the other outcome may reset (c, d) and increase u back to k .

Finally, on $v \leq r$, either $r < m$ and only terminal states occur, or $m \leq r$. In the latter case $k = \lceil \log_2 m \rceil \leq \lceil \log_2 r \rceil$, while (7) gives $0 \leq u = k - c \leq k$. Thus u is bounded in every sublevel set, and Theorem 2 applies. $\square$

This proof isolates the actual AST argument in three facts: m is stable, tree height is logarithmically bounded by m , and at least one child avoids rejection. A direct pCFG treatment would additionally require values at the increment, choice, test, and reset locations, plus cases for both children. FLDR complements the FDR comparison by showing how the program-level rule accommodates rejection and restart through the post-distribution of one iteration.

From AST to total correctness. The reused distributional invariants establish that the terminating FDR output is uniform and that FLDR returns i with probability a_i/m [?]. Theorems 5 and 6 show AST, i.e. that the corresponding post-distributions have total mass one. Hence the partial-correctness distributions are proper output distributions, yielding total correctness for both algorithms.

8.3 Han–Hoshi Sampling: dyadic refinement without tail bounds

The Han–Hoshi sampler [?] draws from a discrete distribution with positive integer weights $a_1, \dots, a_n$ and total weight $m = \sum_{i=1}^n a_i$. Write $S_i = \sum_{j=1}^i a_j$, with $S_0 = 0$. The algorithm repeatedly refines a dyadic interval until it lies entirely inside one of the target intervals

$$I_i = \left[\frac{S_{i-1}}{m}, \frac{S_i}{m} \right), \quad 1 \leq i \leq n.$$

Its sampling loop is:

```

v := 1; c := 0; r := 0;
while (r = 0) do
  v := 2v;
  c := 2c [1/2] c := 2c + 1;
  r := idx(c, v)
end

```

Here

$$\text{idx}(c, v) := \begin{cases} i, & \text{if } S_{i-1}v \leq cm \text{ and } (c+1)m \leq S_i v \text{ for some } i \in \{1, \dots, n\}, \\ 0, & \text{if no such } i \text{ exists.} \end{cases}$$

The pair (c, v) represents the interval

$$J(c, v) = \left[\frac{c}{v}, \frac{c+1}{v} \right).$$

At loop heads the reachable states satisfy

$$v > 0, \quad 0 \leq c < v, \quad 0 \leq r \leq n. \quad (9)$$

Moreover, after an iteration, $r = 0$ exactly when the selected child interval is not contained in a target interval. As in the preceding case studies, a stronger distributional invariant used for output correctness may be retained; only the support information above is needed for AST.

Let B_{HH} denote the loop body. For $b \in \{0, 1\}$, set

$$c_b = 2c + b, \quad v' = 2v, \quad r_b = \text{idx}(c_b, v').$$

For a Dirac input δ_σ ,

$$\text{sp}[[B_{\text{HH}}]](\delta_\sigma) = \frac{1}{2}\delta_{\sigma[v/v', c/c_0, r/r_0]} + \frac{1}{2}\delta_{\sigma[v/v', c/c_1, r/r_1]}. \quad (10)$$

Thus one complete program-level step is precisely a fair choice between the left and right halves of $J(c, v)$.

Theorem 7 (AST of Han–Hoshi). *For every finite vector of positive integer weights, the Han–Hoshi sampling loop terminates almost surely.*

Proof. For a reachable state σ , let

$$N(\sigma) = \left| \left\{ i \in \{1, \dots, n-1\} \mid \frac{c}{v} < \frac{S_i}{m} < \frac{c+1}{v} \right\} \right|$$

be the number of internal target boundaries lying strictly inside $J(c, v)$. Define the two certificates

$$V(\sigma) = U(\sigma) = \begin{cases} 0, & r \neq 0, \\ \max\{1, N(\sigma)\}, & r = 0. \end{cases}$$

They vanish exactly on terminal states, and U is natural-valued.

Consider an enabled state and write $q = N(\sigma)$. Splitting $J(c, v)$ into its two children cannot create an internal target boundary. Every boundary counted in a child is counted in the parent, while a boundary at the common endpoint of the children is counted in neither. If N_b denotes the number of boundaries strictly inside child b , then

$$N_0 + N_1 \leq q. \quad (11)$$

A child remains enabled exactly when it contains an internal boundary in its interior; hence its post-state certificate is N_b if $N_b > 0$ and zero otherwise. Using (9) and (10), for $q > 0$ we obtain

$$\mathbb{E}_\sigma[V'] = \frac{1}{2}(N_0 + N_1) \leq \frac{q}{2} \leq V(\sigma).$$

If $q = 0$, both children are terminal and the expectation is zero. This proves the supermartingale obligation of Theorem 2.

For the decrease obligation, if $q > 0$, at least one of N_0, N_1 is strictly smaller than q ; otherwise their sum would be at least $2q$, contradicting (10). The corresponding post-state therefore has strictly smaller U . It is selected with probability $1/2$, so the decrease condition holds with $\varepsilon = 1/2$. If $q = 0$, both children terminate and decrease U from 1 to 0, so the same bound applies.

Finally, $U = V$ on every reachable state. Consequently, on each sublevel set $V \leq R$ we have $U \leq R$. All obligations of Theorem 2 hold, and the loop is AST. $\square$

The proof replaces the explicit depth- t tail calculation by one iteration-level fact: bisecting an interval distributes its internal target boundaries over the two children, and at least one child contains strictly fewer. The number of such boundaries serves simultaneously as the supermartingale and the natural-valued variant (with the value 1 for the enabled boundary-free initial case). Hence Han–Hoshi supplies a further application of the program-level rule of Theorem 2, with a certificate based on interval refinement rather than the progress-and-restart structure of FDR and FLDR.

9 Related Work

Proof rules for almost-sure termination. Classical bounded-variant arguments establish AST by requiring a uniformly positive probability of descent within a bounded natural-valued range [?]. Supermartingale-based rules extend the scope of this approach. McIver et al. [?] formulate an AST rule directly over source programs, including demonic choice, using one real-valued function with both supermartingale and progress obligations. Majumdar and Sathiyarayanana [?] establish sound and relatively complete proof rules over pCFGs and also prove relative completeness of the earlier rule of McIver et al.

The two-certificate rule of Majumdar and Sathiyarayanana is our starting point: it already separates the supermartingale from the natural-valued variant. We transfer this structure to complete loop iterations in distribution-transformer semantics and relate the resulting certificates to pCFG certificates in both directions. Relative to their rule, the contribution is the program-level formulation, its metatheory, and the reduction in location-dependent proof obligations. Relative to McIver et al., the distinction is the use of two separately chosen functions, coupled through sublevel boundedness, instead of requiring one function to serve both roles. This permits a variant to reset on rejection without itself satisfying a supermartingale inequality. Our FDR comparison makes the change in certificate representation explicit; FLDR and Han–Hoshi illustrate applications of the

source-level rule. We address qualitative AST, which permits infinite expected runtime.

Operational and denotational models. pCFGs and Markov decision processes are the standard operational substrate for termination and reachability analysis of probabilistic and randomized systems [?,?]. Tools such as Storm [?] and refined analyses of visiting times and stationary behavior [?,?] operate at this level. Viewing probabilistic systems as *distribution transformers* rather than as single-state transition systems has recently been fruitful for both Markov chains and MDPs: Aghamov et al. [?,?] model-check Markov chains through their action on distributions, and Akshay et al. [?,?] synthesize invariants and certified policies for MDPs under distributional reach-avoidance objectives; distribution-based objectives for MDPs are also studied in [?]. Our rule applies the same distribution-transformer perspective one level higher, directly to program syntax rather than to an underlying automaton, which avoids the location-indexed bookkeeping that a translation to pCFGs or MDPs otherwise imposes.

Denotational semantics and program logics for probabilistic programs. The denotational, post-distribution-transformer semantics we build on originates with Kozen [?,?] and underlies the weakest-precondition and weakest pre-expectation calculi of McIver and Morgan [?] and Kaminski [?], in the tradition of Dijkstra’s predicate-transformer semantics [?,?]. Hoare-style program logics for probabilistic programs were introduced by den Hartog and de Vink [?] and extended by the Ellora framework [?]; more recent calculi include quantitative strongest-postcondition reasoning [?], its hyperproperty generalization [?], and modal separation logic for conditional probability [?]. These frameworks provide broader forms of reasoning about probabilistic behavior. Our focus is a specific AST rule that can be used alongside distributional partial-correctness arguments.

Verifying exact sampling algorithms. Exact discrete samplers built from fair coin flips, following the classical constructions of von Neumann [?] and Lumbroso’s Fast Dice Roller (FDR) [?], have received growing verification attention. Saad et al. introduce the Fast Loaded Dice Roller (FLDR) as a near-optimal sampler for finite discrete distributions [?]. Han and Hoshi’s interval algorithm [?] supplies the basis for our third sampler case study. Bagnall, Stewart, and Banerjee’s Zar compiler [?,?] formally verifies samplers compiled from probabilistic programs with loops and conditioning in Coq, targeting the random-bit model; generating-function techniques have likewise been used to reason about probabilistic programs and about samplers such as the fast dice roller [?,?,?]. Exact and guaranteed-bound inference engines such as PSI [?,?], the approach of Beutner et al. [?], and scalable exact inference for discrete programs [?] address the complementary problem of computing or bounding the output distribution.

Closest to our work is the distributional-invariant framework of Zilken et al. [?]. It already establishes partial and total correctness of FDR and FLDR, with AST arguments using existing rules in its appendices. We build on those invariants and verification examples to give a general program-level AST rule and

an explicit comparison of proof representations. The Han–Hoshi case study adds a different sampling mechanism, using interval boundaries as the termination certificate. Its role here is to demonstrate applicability of the AST rule; the present manuscript does not develop a separate distributional output-correctness proof for Han–Hoshi.

Nondeterminism. Several lines of work verify probabilistic programs or Markov models in the presence of demonic or angelic nondeterminism [?, ?, ?, ?, ?]. Demonic AST requires termination with probability one under every admissible scheduler; both McIver et al. [?] and the pCFG framework of Majumdar and Sathiyarayanana [?] support this interpretation. Our rule instead uses a single post-distribution for each input and currently excludes nondeterministic choice. Extending it requires a strategy-indexed or set-valued semantics and corresponding invariant and progress obligations. Strategy synthesis, as studied by Batz et al. [?], offers a complementary route for proving AST under a selected strategy. We discuss the precise obstacles and possible extensions in Section 10.

10 Conclusion and Future Work

Achieved results. We formulated the two-certificate AST approach of Majumdar and Sathiyarayanana directly over the distribution-transformer semantics of probabilistic while-loops with loop-free bodies. The supermartingale and variant describe complete loop iterations, so their use does not require intermediate control-location annotations. We established soundness and relative completeness for this fragment through translations between program-level and pCFG-level certificates.

The examples serve complementary purposes. The one-dimensional random walk introduces the rule and its ability to handle infinite expected runtime. FDR provides an explicit comparison of pCFG-level and program-level AST proofs: the latter eliminates the location-dependent cases, scaling factors, and offsets of the former while reusing the distributional invariant. FLDR demonstrates how a variant can decrease on progress and reset on rejection, with a separate supermartingale controlling its sublevel bounds. Han–Hoshi demonstrates a different certificate construction: the number of target boundaries in the current dyadic interval captures progress under refinement. For FDR and FLDR, our AST proofs integrate with existing distributional output-correctness arguments [?]. These case studies support the benefit of expressing termination arguments over complete iterations; the explicit comparison of proof representations is provided by FDR.

Limitations. The rule applies to a single while-loop with a *loop-free* body and no nondeterministic choice. Loop-freeness guarantees that each body execution terminates and has bounded control-flow length, as used by our certificate translations. The denotational transformer itself is also defined for bodies containing loops; extending the rule to them requires suitable termination summaries and

a revised metatheory. We also restrict attention to rational-valued variables and discrete probabilistic choices.

Nondeterminism: which termination property? Under the standard demonic interpretation, AST means termination with probability one under *every* admissible scheduler [?, ?]. Writing $\Pr_\sigma^\pi(C\downarrow)$ for the termination probability from state σ under scheduler π , this means

$$\forall \sigma \in \text{supp}(M) \forall \pi : \Pr_\sigma^\pi(C\downarrow) = 1,$$

equivalently $\inf_\pi \Pr_\sigma^\pi(C\downarrow) = 1$ for every such σ . Here schedulers may use the finite execution history and randomization to resolve enabled choices; no fairness restriction is assumed. Proving AST under one fixed scheduler, or the existence of an almost-surely terminating scheduler, is a different objective. In particular, in infinite-state models a supremal termination probability of one need not be attained by any scheduler [?]. The pCFG framework of Majumdar and Sathiyarayanan already handles bounded demonic nondeterminism; our current specialization does not transfer that capability to the program-level rule.

Where the present formulation stops. Our premises use the unique distribution $\text{sp}[B](\delta_\sigma)$. With nondeterministic choice, different resolutions can produce different post-distributions, so the current semantics has no clause for that construct. Natural alternatives are a strategy-indexed semantics or a semantics collecting possible post-distributions. If a chosen resolution is expressible as a program in our language with a loop-free body, the current rule can already prove AST of that resolved program, hence AST under the selected strategy. Fixing a history-dependent scheduler does not automatically give a reusable transformer on distributions over program states alone: the scheduler's residual memory must also be accounted for when composing iterations.

Pointwise minima or maxima of output distributions do not solve this problem. For example, the nondeterministic choice between $x := 0$ and $x := 1$, from a fixed initial state, yields two distinct Dirac distributions. Their pointwise minimum has mass zero and their pointwise maximum has mass two, although both branches terminate. Thus neither construction supplies the required output-distribution semantics. Infima and suprema are instead useful for bounding scalar expectations or event probabilities over possible distributions.

The proof obligations must change as well. For demonic AST, the invariant must be preserved by every admissible one-iteration behavior, and both certificate inequalities must hold for every such behavior. To indicate a possible formulation, let $\mathcal{K}_B(\sigma)$ collect the output distributions of the loop-free body from σ under all within-iteration schedulers. With common certificates v, u , the natural robust obligations are

$$\begin{aligned} \sup_{\nu \in \mathcal{K}_B(\sigma)} \sum_{\sigma'} \nu(\sigma') v(\sigma') &\leq v(\sigma), \\ \inf_{\nu \in \mathcal{K}_B(\sigma)} \sum_{\sigma' : u(\sigma') < u(\sigma)} \nu(\sigma') &\geq \varepsilon. \end{aligned}$$

These would be required at every enabled invariant state, with one $\varepsilon > 0$ independent of the state and the resolution of choice, alongside invariant closure, the terminal-state conditions, and sublevel boundedness. Checking only a favorable resolution cannot establish demonic AST: a loop that nondeterministically chooses between exiting and skipping can be kept running forever by always skipping. For an existential objective, one consistent strategy must satisfy the invariant and both certificate obligations; optimizing them separately need not produce the same strategy. The displayed obligations are proposed as a direction for future work; Theorem 2 currently establishes only the purely probabilistic case. Our contraction and lifting arguments would also need to account for non-deterministic choices. A compositional semantics and corresponding soundness and relative-completeness results remain future work.

Certificate synthesis. Automating certificate construction is a next step. The one-iteration transformer exposes constraints for the supermartingale and progress obligations, suggesting template-based searches for v and u that reuse a supplied invariant. A synthesis procedure must also enforce the terminal-state conditions, natural-valuedness of u , a uniform positive progress bound, and boundedness of u on every sublevel set of v ; the two searches therefore remain coupled. Establishing soundness of the generated constraints and evaluating how much certificate construction can be automated are open tasks. The present case studies use manually supplied certificates.

Further directions. A fully compositional treatment of nested loops would allow AST summaries to be combined along a program’s syntax tree. Strategy-based semantics and programmatic strategy synthesis [?] provide starting points for the nondeterministic extension outlined above. A comparison with the combined certificate of McIver et al. [?] could identify program classes for which either style is easier to construct. A broader evaluation could quantify certificate cases, verification conditions, and proof-development effort beyond the FDR comparison.

A Translation and AST Preservation

Well-defined loop semantics. The transformer domain $\mathcal{H}$ from Section 2 has pointwise suprema of increasing ω -chains: these preserve ω -continuity and the mass bound by monotone convergence. Its least element is $\perp = \lambda\eta.0$. By structural induction, the body transformer $\text{sp}[B]$ is ω -continuous and does not increase mass. For $f \in \mathcal{H}$ and $\eta \in \Delta_{\leq 1}\Sigma$,

$$|\Phi_{b,B}(f)(\eta)| = |[-b]\eta| + |f(\text{sp}[B]([b]\eta))| \leq |[-b]\eta| + |[b]\eta| = |\eta|.$$

Composition and addition preserve ω -continuity here, so $\Phi_{b,B}(f) \in \mathcal{H}$. Moreover, for every increasing chain $(f_n)_n$ in $\mathcal{H}$, pointwise evaluation gives $\Phi_{b,B}(\sup_n f_n) = \sup_n \Phi_{b,B}(f_n)$. Kleene’s theorem therefore yields ???. Structural induction also establishes linearity on sums that are subdistributions: in the loop case, every approximant is linear, and its pointwise supremum remains linear. The countable-sum property follows from ω -continuity.

Operational correspondence. The structural translation $\mathcal{T}$ introduces entry and exit locations for every command. Assignment edges update the valuation, guarded edges encode conditionals and loop tests, and a probabilistic-choice location has outgoing probabilities p and $1 - p$. Sequential composition identifies the first command's exit with the second command's entry.

Lemma 3 (Loop-free correspondence). *For loop-free B and states σ, σ' ,*

$$\text{sp}[[B]](\delta_\sigma)(\sigma') = \Pr(\text{Paths}_{\mathcal{T}[B]}((\ell_{\text{in}}, \sigma), (\ell_{\text{out}}, \sigma'))).$$

Proof. By structural induction on B . Skip and assignment have one path. Sequential composition uses convolution over intermediate states. Conditional paths are partitioned by the guard. Probabilistic choice multiplies branch-path probabilities by p and $1 - p$, matching linearity of the post-transformer. $\square$

For a loop $C = \text{while } (b) \{B\}$ with loop-free body, induction on j shows that $\Phi_{b,B}^j(\perp)(\mu)$ is the subdistribution of pCFG runs that exit within at most j visits to the outer loop guard. The zeroth approximant is zero; for $j \geq 1$, this means termination after at most $j - 1$ completed body iterations. The induction step partitions runs into those that exit at the first guard test and those that execute B and then terminate within the remaining guard visits. Taking pointwise suprema gives equality between post-distribution weight and pCFG reachability probability. Therefore $|\text{sp}[[C]](\delta_\sigma)| = 1$ iff the translated graph reaches its exit almost surely from $(\ell_{\text{in}}, \sigma)$. Linearity and ω -continuity extend the equivalence to arbitrary initial subdistributions.

B Detailed One-Iteration Calculations

FDR. Starting in σ , doubling v and sampling the bit produces the two Dirac states $\sigma[v/2v, c/2c]$ and $\sigma[v/2v, c/2c + 1]$, each of mass $1/2$. Filtering by $c \geq n$ and applying both subtraction assignments yields Equation (6). On the left-bit outcome, subtraction of n from both variables preserves their difference, hence in all enabled cases

$$(v' - c') = 2(v - c).$$

This is the algebraic core of the FDR variant argument.

FLDR. After incrementing c , the fair choice reaches indices $2d$ and $2d + 1$. A rejecting child maps to $(c, d) = (0, 0)$; a non-rejecting child retains depth $c + 1$. Since preprocessing never places the unique rejecting outcome in both children of one internal node, the total mass of non-reset successors is at least $1/2$. Every such successor reduces $k - c$ by one.

C Proof-Obligation Comparison

The comparison below summarizes the forms of proof obligations used in the two FDR proofs. It is a qualitative comparison of their representation, not a measurement of proof-development or solver runtime.

	pCFG level	program level
Invariant	location-indexed state predicate	one distributional invariant
Supermartingale	edge/choice obligations	one expected body-step inequality
Variant	edge-wise strict/probabilistic descent	one post-distribution inequality
Control-flow constants	scaling and location offsets	none
Reuse from partial correctness	requires projection/lifting	direct

For FDR, both columns are feasible and prove the same AST fact. This makes FDR a controlled comparison of proof style. The program-level proofs of FLDR and Han–Hoshi further demonstrate how progress is expressed using non-reset outcomes and interval boundaries, respectively; they are not additional comparisons against explicitly constructed pCFG certificates.

D Artifact and Proof-Development Notes

The current manuscript is a venue-independent draft. Before submission, the following items should be completed: (i) mechanize or fully expand the translation-correctness proof; (ii) state the preprocessing properties of FLDR as explicit lemmas and connect them to its reference implementation; (iii) decide whether the generalized ε_r version belongs in the main theorem; and (iv) evaluate proof size using a stable metric, for example number of certificate cases and generated verification conditions. The anonymous author block and review options should be replaced only after a venue has been selected.